Name: Naveen Kumar P **Reg. No.:** 192412326

Experiment No.: 5

Experiment Question

Write a program for Implementation of K-Nearest Neighbours (K-NN) in Python.

Aim

To implement the K-Nearest Neighbours algorithm in Python, use it to classify a new sample, and evaluate its accuracy for different values of K.

Concept

K-Nearest Neighbours (K-NN) is a simple, supervised, instance-based (lazy) learning algorithm. It does not build a model during training; it stores the whole training set. To classify a new sample, it:

- calculates the distance from the new sample to every training sample,
- picks the K training samples that are closest (the K nearest neighbours),
- assigns the class that occurs most often among those K neighbours (majority vote).

The value of K affects the result: a very small K is sensitive to noise, while a very large K blurs the class boundaries. An odd K is usually chosen for two-class problems to avoid ties.

Dataset Description

A T-shirt size dataset is used, stored in a CSV file named `tshirt.csv`. It has 18 samples, 2 numeric features (Height in cm, Weight in kg) and 1 target class (T-shirt size: M = Medium, L = Large). There are 7 samples of class M and 11 of class L.

Height (cm)	Weight (kg)	T-shirt size
158	58	M
158	59	M
158	63	M
160	59	M

Height (cm)	Weight (kg)	T-shirt size
160	60	M
163	60	M
163	61	M
160	64	L
163	64	L
165	61	L
165	62	L
165	65	L
168	62	L
168	63	L
168	66	L
170	63	L
170	64	L
170	68	L

Contents of `tshirt.csv`:

```
Height,Weight,TShirtSize
158,58,M
158,59,M
158,63,M
160,59,M
160,60,M
163,60,M
163,61,M
160,64,L
163,64,L
165,61,L
165,62,L
165,65,L
168,62,L
168,63,L
```

```
168,66,L
170,63,L
170,64,L
170,68,L
```

New sample to classify: Height = 161 cm, Weight = 61 kg, with K = 5.

Algorithm

```
K-NN Algorithm
1. Load the training data and choose the value of K.
2. For the new sample x:
    a. Calculate the distance between x and every training sample.
    b. Sort the distances in ascending order.
    c. Select the first K training samples (the K nearest neighbours).
    d. Count the number of neighbours in each class.
    e. Assign to x the class with the highest count.
3. Output the predicted class.
```

Step-by-step Procedure

1. Read the training data from `tshirt.csv`.
2. Define a function to calculate the Euclidean distance between two points.
3. Define the K-NN function: compute the distance from the query point to every training point, sort them, and take the K smallest.
4. Find the majority class among the K neighbours using a vote count.
5. Classify the new sample (161, 61) with K = 5 and print its neighbours and the predicted size.
6. Evaluate the model by leave-one-out testing: each sample is removed from the training set in turn and predicted from the rest. Repeat for K = 1, 3, 5 and 7.
7. Compare the accuracy for the different values of K.

Formulae

Euclidean distance between a query point q and a training point p with n features:

```
d(p, q) = \sqrt{\sum_{i=1}^{n} (p_i - q_i)^2}
```

Example: distance from the new sample (161, 61) to the training sample (160, 60):

```
d = \sqrt{(161 - 160)^2 + (61 - 60)^2} = \sqrt{2} = 1.4142
```

Predicted class (majority vote over the set N_K of the K nearest neighbours):

```
\hat{y} = \arg\max_{c} \sum_{i \in N_K} \mathbf{1}(y_i = c)
```

Accuracy:

```
Accuracy = \frac{\text{number of correct predictions}}{\text{total number of predictions}} \times 100
```

Python Program

```
import csv
import math
from collections import Counter

# Step 1: Load the dataset
with open('tshirt.csv') as f:
    reader = csv.reader(f)
    header = next(reader)
    data = [(float(r[0]), float(r[1]), r[2]) for r in reader]

# Euclidean distance between two points
def euclidean(p, q):
    return math.sqrt((p[0] - q[0]) ** 2 + (p[1] - q[1]) ** 2)

# K-NN classifier
def knn_predict(train, query, k):
    distances = [(euclidean(query, (h, w)), label) for h, w, label in train]
    distances.sort(key=lambda d: d[0])
    neighbours = distances[:k]
    votes = Counter(label for _, label in neighbours)
    return votes.most_common(1)[0][0], neighbours

# Step 2: Classify a new sample
new_sample = (161, 61)
k = 5
prediction, neighbours = knn_predict(data, new_sample, k)

print(f"New sample (Height, Weight): {new_sample},  K = {k}")
print(f"{k} nearest neighbours:")
for dist, label in neighbours:
    print(f"  distance = {dist:.4f}  class = {label}")
print("Predicted T-shirt size:", prediction)

# Step 3: Evaluate accuracy using leave-one-out testing for different K
print("\nLeave-one-out accuracy:")
for k in [1, 3, 5, 7]:
    correct = 0
    for i, (h, w, label) in enumerate(data):
        train = data[:i] + data[i + 1:]
        pred, _ = knn_predict(train, (h, w), k)
        if pred == label:
```

```
            correct += 1
    print(f"  K = {k}: {correct}/{len(data)} correct, accuracy = {correct / len(data) * 100:.2f}%")
```

Output

```
New sample (Height, Weight): (161, 61),  K = 5
5 nearest neighbours:
  distance = 1.4142  class = M
  distance = 2.0000  class = M
  distance = 2.2361  class = M
  distance = 2.2361  class = M
  distance = 3.1623  class = L
Predicted T-shirt size: M

Leave-one-out accuracy:
  K = 1: 16/18 correct, accuracy = 88.89%
  K = 3: 13/18 correct, accuracy = 72.22%
  K = 5: 16/18 correct, accuracy = 88.89%
  K = 7: 15/18 correct, accuracy = 83.33%
```

Result / Conclusion

The K-NN algorithm was implemented in Python. For the new sample (Height = 161 cm, Weight = 61 kg) with K = 5, four of the five nearest neighbours belong to class M, so the predicted T-shirt size is **Medium (M)**. In leave-one-out testing, K = 1 and K = 5 gave the highest accuracy (88.89%), while K = 3 gave the lowest (72.22%). Hence the K-Nearest Neighbours algorithm was successfully implemented and demonstrated.